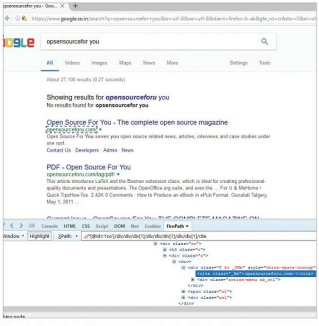

Figure 1: Firepath and Firebug being used to identify the UI element locator for selenium

```
    <dependency>
        <groupId>com.github.2gis.winium</groupId>
        <artifactId>winium-webdriver</artifactId>
        <version>0.1.0-1</version>
    </dependency>

  </dependencies>
  </project>
```

Once we have the above requirements set up on the desktop, let's begin initiating the Winium driver, which is similar to the procedure for initiating the Selenium driver, as shown below.

We can write our test in any language that's compatible with WebDriver such as Java, Objective-C, JavaScript with Node.js PHP, Python, Ruby, C#, Perl with the Selenium WebDriver API and language-specific client libraries. I have written it in Java, as shown below:

```
DesktopOptions option = new DesktopOptions();

option.setApplicationPath("C:\\Windows\\System32\\calc.exe");

WiniumDriver driver = new WiniumDriver(new URL("http://
localhost:9999"), option);
```

I have created the object of DesktopOptions, which will help us to point the application we are automating (I have used the calculator application for explaining this).

By initiating the Winium driver using the *Winiumdriver*

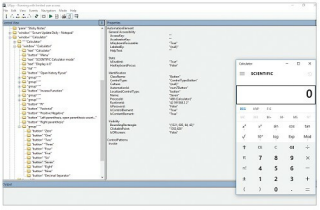

Figure 2: UISpy showing the properties of the button 'seven' of the calculator used to find the element properties for Winium

class, we are passing the Winium server URL and the path of our desktop application on which automation is intended to be carried out.

The server URL is obtained by clicking on the *Winium.desktop.driver.exe* that was downloaded.

The process of initialising the Winium driver is similar to that of the Selenium driver, as we see with reference to Selenium.

```
WebDriver driver = new FirefoxDriver();
```

Now we are all set to write some code to interact with the desktop application. I have used it to open the calculator and to perform the task of adding seven and eight, and to capture the result.

```
driver.findElement(By.name("Seven")).click();
driver.findElement(By.name("Plus")).click();
driver.findElement(By.name("Eight")).click();
driver.findElement(By.name("Equals")).click();
Thread.sleep(5000);

String output = driver.findElement(By.
id("CalculatorResults")).getAttribute("Name");

System.out.println("Result after addition is: "+output);
```

We can identify the element using the name, ID and *xpath*, just as we do using Firepath in Selenium.

Please refer the complete working code mentioned below:

```
package com.winium.demo;

import java.net.MalformedURLException;
import java.net.URL;

import org.openqa.selenium.By;
```